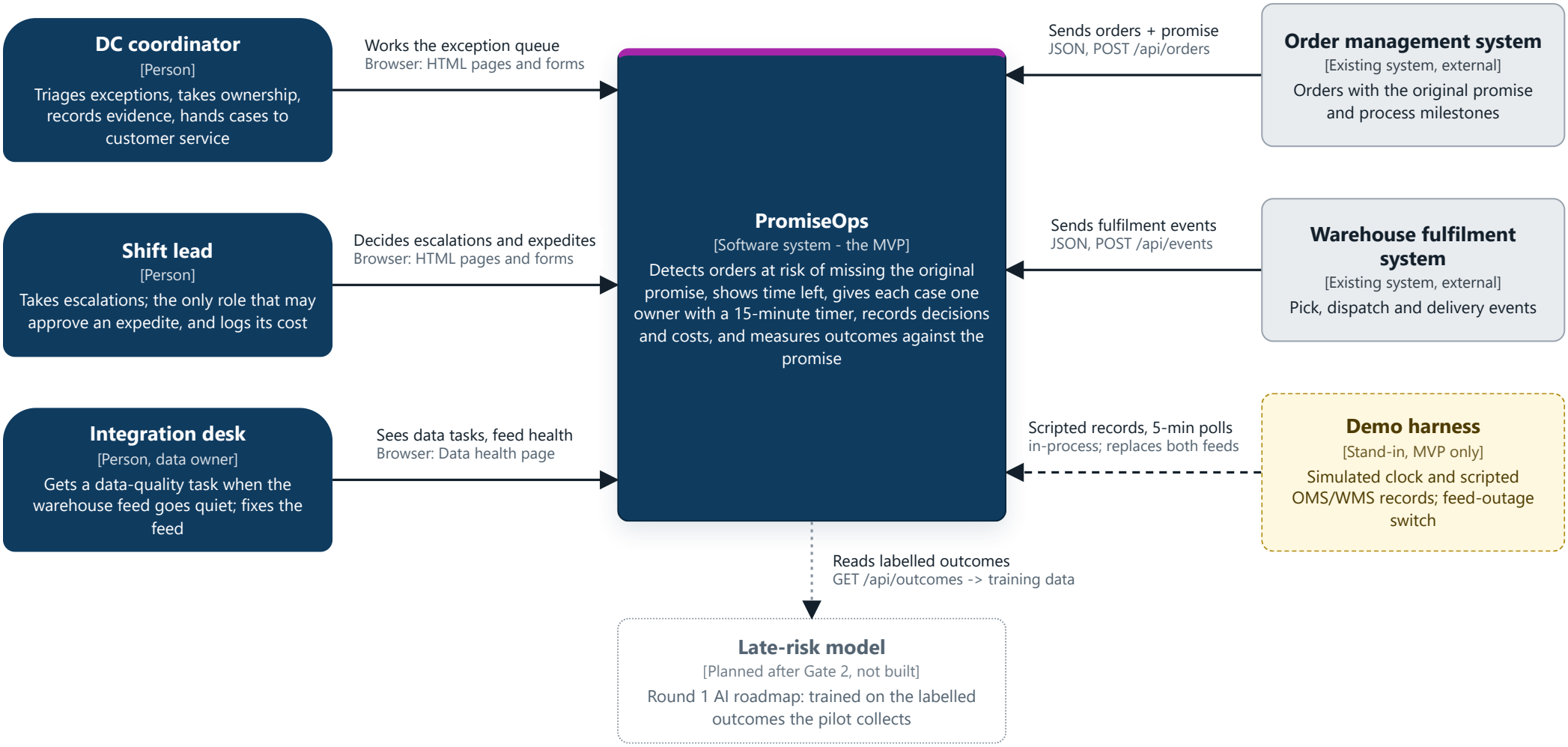
System context: what is inside PromiseOps and what is outside

C4 level 1. Every solid arrow is an interaction the MVP performs; dashed is the demo stand-in, dotted is planned.



Inside the boundary

Two exception rules, priority queue, ownership and escalation, evidence and cost on every decision, audit log, data-freshness guard, outcome measurement against the Round 1 pilot targets, English and Vietnamese UI.

Outside, read only

Order and warehouse systems keep owning orders, stock and dispatch. PromiseOps never writes back; if it is down, staff use the current floor process. API calls need an X-API-Key when one is configured.

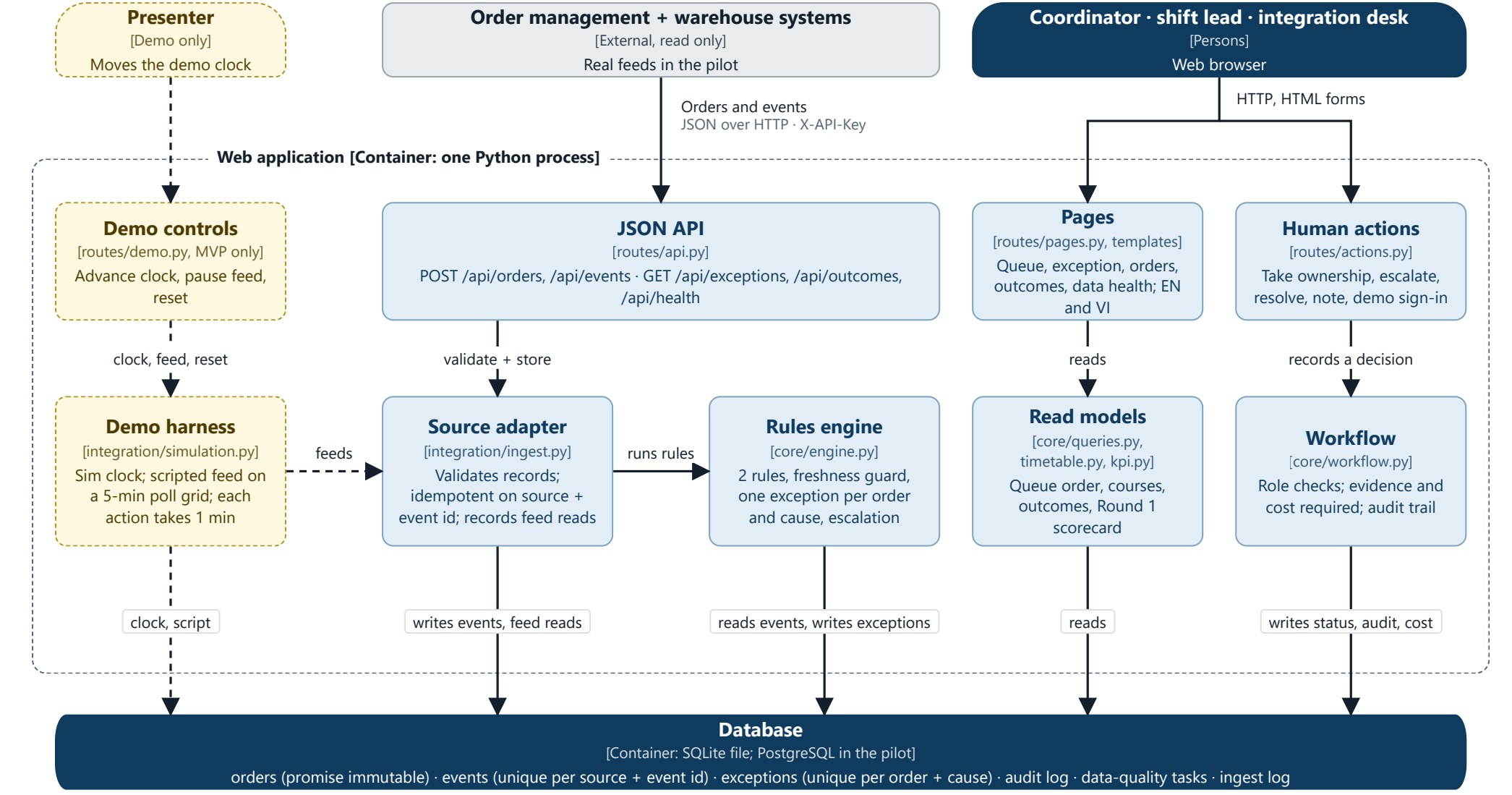
Deliberately not in the MVP

- Write-back, routing, replenishment, promise changes
- Customer messaging: "handed to customer service" is recorded, not sent
- AI inside the product: rules first, as argued in Round 1
- Company SSO: the demo uses a role switch

Person · System in scope · Existing external system · Demo stand-in (MVP only) · Planned, not built

Inside PromiseOps: one container, three layers, one database

C4 levels 2 and 3. One Python service with clear modules, as proposed for the pilot; no microservices.



1 · READ
The adapter reads each feed every 5 min and records that the read succeeded.

2 · CHECK
Duplicates ignored, bad records rejected. No read after a milestone, or a feed older than 30 min: alerts pause, a data task opens.

3 · DETECT
Two rules raise one exception per order and cause, owned by the coordinator, 15-min timer, then escalation.

4 · ACT
People decide. Expedite only by the shift lead; evidence on every outcome, cost on every expedite.

5 · PROVE
Outcomes against the original promise; Round 1 scorecard; labelled data for the planned risk model.

Component (module named) · Demo only, replaced by real feeds and time in the pilot · Person / database · Same READ, CHECK, ACT, PROVE stages as Round 1, slide 5